EduSync Academy / WAQAR INSTITUTE - Complete Architecture PDF

 # EduSync Academy / WAQAR INSTITUTE Complete Production Security & Direct APK Build Architecture

This document contains the complete, end-to-end production code from top to bottom covering all **15 Direct Sideloaded APK Security Tasks** and the **5 Enterprise Performance Libraries**:
- `/android-kotlin/app/build.gradle.kts`
- `/android-kotlin/app/proguard-rules.pro`
- `/android-kotlin/app/src/main/java/com/edusync/academy/security/EduSyncSecuritySuite.kt`
- `/functions/src/index.ts`
- `/firestore.rules`
- `/.github/workflows/build-apk.yml`

1. Gradle Build Configuration & 5 Enterprise Performance Libraries (`/android-kotlin/app/build.gradle.kts`)

Includes **AndroidX ProfileInstaller (`1.4.1`)**, **Coil 3 (`3.0.4`)**, **Kotlinx Immutable Collections (`0.3.8`)**, **AndroidX Paging 3 (`3.3.4`)**, **AndroidX DataStore (`1.1.1`)**, and environment-driven `signingConfigs` harmonized with **Task 2** and **Task 15**:

```kotlin
import java.io.FileInputStream
import java.util.Properties

plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
    id("org.jetbrains.kotlin.plugin.serialization")
    id("com.google.devtools.ksp")
}

android {
    namespace = "com.edusync.academy"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.edusync.academy"
        minSdk = 26
        targetSdk = 35
        versionCode = 12
        versionName = "2.4.0-direct"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
        vectorDrawables {
            useSupportLibrary = true
        }

        val releaseCertSha256 = System.getenv("RELEASE_CERT_SHA256")
            ?: "E3B0C44298FC1C149AFBF4C8996FB92427AE41E4649B934CA495991B7852B855"
        val debugCertSha256 = System.getenv("DEBUG_CERT_SHA256")
            ?: "A40DA80A59D170CAA950CF15C18C454D47A39B26989D8B640ECD745BA71BF5DC"

        buildConfigField("String", "EXPECTED_RELEASE_CERT_SHA256", "\"$releaseCertSha256\"")
        buildConfigField("String", "EXPECTED_DEBUG_CERT_SHA256", "\"$debugCertSha256\"")
        buildConfigField("String", "FIREBASE_PROJECT_ID", "\"adroit-graph-h2w8r\"")
        buildConfigField("String", "FIRESTORE_DATABASE_ID",
"\"ai-studio-988a7681-f0d0-4922-93ac-38c131c1cc37\"")
    }

    signingConfigs {
        create("release") {
            val keystorePath = System.getenv("KEYSTORE_FILE_PATH") ?: "release-keystore.jks"
            val keystoreFile = file(keystorePath)
            if (keystoreFile.exists()) {
                storeFile = keystoreFile
                storePassword = System.getenv("KEYSTORE_PASSWORD") ?: ""
                keyAlias = System.getenv("KEY_ALIAS") ?: ""
                keyPassword = System.getenv("KEY_PASSWORD") ?: ""
                enableV1Signing = true
                enableV2Signing = true
                enableV3Signing = true
                enableV4Signing = false
            }
```

```
        }
    }

    lint {
        abortOnError = false
        checkReleaseBuilds = false
    }

    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            val releaseSigning = signingConfigs.findByName("release")
            if (releaseSigning?.storeFile?.exists() == true) {
                signingConfig = releaseSigning
            } else {
                signingConfig = signingConfigs.getByName("debug")
            }
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
        debug {
            isMinifyEnabled = false
            signingConfig = signingConfigs.getByName("debug")
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
        freeCompilerArgs = freeCompilerArgs + listOf(
            "-opt-in=androidx.compose.material3.ExperimentalMaterial3Api",
            "-opt-in=kotlinx.coroutines.ExperimentalCoroutinesApi"
        )
    }

    buildFeatures {
        compose = true
        buildConfig = true
    }

    packaging {
        resources {
            excludes += "/META-INF/{AL2.0,LGPL2.1}"
        }
    }
}

dependencies {
    // ZERO GOOGLE PLAY STORE DEPENDENCIES:
    // No com.google.android.play:integrity, no Play In-App Updates, no LVL.

    val composeBom = platform("androidx.compose:compose-bom:2024.10.01")
    implementation(composeBom)

    // Firebase v33+ Modular BOM (Auth, Firestore, Functions, Storage)
    val firebaseBom = platform("com.google.firebase:firebase-bom:33.6.0")
    implementation(firebaseBom)
    implementation("com.google.firebase:firebase-auth")
    implementation("com.google.firebase:firebase-firestore")
    implementation("com.google.firebase:firebase-functions")
    implementation("com.google.firebase:firebase-storage")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.9.0")
```

```
    // 1. AndroidX ProfileInstaller (AOT Baseline Profiles for Sideloaded APKs)
    implementation("androidx.profileinstaller:profileinstaller:1.4.1")

    // 2. Coil 3 (Async Non-Blocking Image Loading & Caching)
    implementation("io.coil-kt.coil3:coil-compose:3.0.4")
    implementation("io.coil-kt.coil3:coil-network-okhttp:3.0.4")

    // 3. Kotlinx Immutable Collections (Zero-Recomposition Lists)
    implementation("org.jetbrains.kotlinx:kotlinx-collections-immutable:0.3.8")

    // 4. AndroidX Paging 3 (Low-Heap Reactive Pagination)
    implementation("androidx.paging:paging-runtime-ktx:3.3.4")
    implementation("androidx.paging:paging-compose:3.3.4")

    // 5. AndroidX DataStore (Coroutine Flow State Persistence)
    implementation("androidx.datastore:datastore-preferences:1.1.1")

    // Jetpack Security Crypto (Hardware Keystore AES256-GCM EncryptedSharedPreferences)
    implementation("androidx.security:security-crypto:1.1.0-alpha06")

    // Core AndroidX & Jetpack Compose UI
    implementation("androidx.core:core-ktx:1.15.0")
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
    implementation("androidx.activity:activity-compose:1.9.3")
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-graphics")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-extended")
    implementation("androidx.navigation:navigation-compose:2.8.4")

    // CameraX & ML Kit Barcode Scanning (On-Device Bundled Model)
    implementation("androidx.camera:camera-core:1.4.0")
    implementation("androidx.camera:camera-camera2:1.4.0")
    implementation("androidx.camera:camera-lifecycle:1.4.0")
    implementation("androidx.camera:camera-view:1.4.0")
    implementation("com.google.mlkit:barcode-scanning:17.3.0")

    // Location Services for Hardware GPS & Mock Location Detection
    implementation("com.google.android.gms:play-services-location:21.3.0")

    // Networking & Serialization
    implementation("com.squareup.okhttp3:okhttp:4.12.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.9.0")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.9.0")

    // Unit Testing
    testImplementation("junit:junit:4.13.2")
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.9.0")
}
```

2. Task 3: Production R8 / ProGuard Full Code Obfuscation Rules (`/android-kotlin/app/proguard-rules.pro`)

```proguard
# ================================================================================
# TASK 3: Production R8 / ProGuard Full Code Obfuscation Rules (EduSync Academy)
# Target: Direct Sideloaded Release APK (./gradlew assembleRelease)
# ================================================================================

# 1. Aggressive Obfuscation & Single-Character Package Flattening
-optimizationpasses 5
-overloadaggressively
-repackageclasses 'o'
-allowaccessmodification
-renamesourcefileattribute SourceFile
-keepattributes SourceFile,LineNumberTable,*Annotation*,Signature,InnerClasses,EnclosingMethod
```

```
# Strip all verbose/debug Android Log calls from production bytecode
-assumenosideeffects class android.util.Log {
    public static boolean isLoggable(java.lang.String, int);
    public static int v(...);
    public static int d(...);
    public static int i(...);
}

# 2. Android Framework & Lifecycle Entry Points
-keep public class * extends android.app.Application
-keep public class * extends androidx.core.content.FileProvider
-keepclassmembers class * extends androidx.lifecycle.ViewModel {
    public <init>(...);
}

# 3. Kotlin Coroutines & Flow Preservation
-keepnames class kotlinx.coroutines.internal.MainDispatcherFactory {}
-keepnames class kotlinx.coroutines.CoroutineExceptionHandler {}
-keepclassmembers class kotlinx.coroutines.** {
    volatile <fields>;
}
-dontwarn kotlinx.coroutines.**

# 4. Kotlinx Serialization & Immutable Collections
-dontnote kotlinx.serialization.SerializationKt
-keepclassmembers class * {
    *** Companion;
}
-keepclasseswithmembers class * {
    kotlinx.serialization.KSerializer serializer(...);
}
-keepclassmembers class * implements kotlinx.serialization.KSerializer {
    java.lang.Object deserialize(...);
    void serialize(...);
}
-keep,allowobfuscation,allowshrinking @kotlinx.serialization.Serializable class * {
    <init>(...);
    void set*(...);
    *** get*();
}
-keep class kotlinx.collections.immutable.** { *; }

# 5. Firebase Firestore, Auth & Cloud Functions Serialization
-keepattributes Signature,InnerClasses,EnclosingMethod
-keep class com.google.firebase.** { *; }
-dontwarn com.google.firebase.**
-keepclassmembers class * {
    @com.google.firebase.firestore.PropertyName <fields>;
    @com.google.firebase.firestore.PropertyName <methods>;
}

# 6. AndroidX Security (EncryptedSharedPreferences / Tink) & DataStore
-keep class com.google.crypto.tink.** { *; }
-dontwarn com.google.crypto.tink.**
-keep class androidx.datastore.** { *; }

# 7. Coil 3 & OkHttp 4 / Retrofit 2
-keep class coil3.** { *; }
-dontwarn coil3.**
-dontwarn okhttp3.**
-dontwarn okio.**
-keep class okhttp3.** { *; }
-keep interface okhttp3.** { *; }
-keepattributes RuntimeVisibleAnnotations,RuntimeVisibleParameterAnnotations
-keepclassmembers,allowshrinking,allowobfuscation interface * {
    @retrofit2.http.* <methods>;
}

# 8. CameraX & ML Kit Barcode Scanning (Bundled Native JNI Models)
-keep class androidx.camera.** { *; }
-keep class com.google.mlkit.** { *; }
-keep class com.google.android.libraries.barhopper.** { *; }
-keepclasseswithmembernames class * {
```

```
    native <methods>;
}
```

---

3. Complete Android Security & Performance Implementation
(`/android-kotlin/app/src/main/java/com/edusync/academy/security/EduSyncSecuritySuite.kt`)

Covers **Tasks 1, 2, 4, 5, 7, 9, 10, 11, 12, 13** plus **ProfileInstaller**, **Coil 3**, **ImmutableList**, **Paging 3**, and **DataStore**:

```kotlin
package com.edusync.academy.security

import android.annotation.SuppressLint
import android.app.Activity
import android.app.AppOpsManager
import android.app.DownloadManager
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.content.SharedPreferences
import android.content.pm.PackageManager
import android.graphics.Bitmap
import android.graphics.pdf.PdfRenderer
import android.location.Location
import android.net.Uri
import android.os.Build
import android.os.Environment
import android.os.ParcelFileDescriptor
import android.os.Process
import android.util.Size
import android.view.ViewTreeObserver
import android.view.WindowManager
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.LinearProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.drawWithContent
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
```

```
import androidx.compose.ui.graphics.drawscope.drawIntoCanvas
import androidx.compose.ui.graphics.nativeCanvas
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalLifecycleOwner
import androidx.compose.ui.platform.LocalView
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.compose.ui.window.Dialog
import androidx.compose.ui.window.DialogProperties
import androidx.core.content.ContextCompat
import androidx.core.content.FileProvider
import androidx.datastore.core.DataStore
import androidx.datastore.preferences.core.Preferences
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.core.intPreferencesKey
import androidx.datastore.preferences.core.stringPreferencesKey
import androidx.datastore.preferences.preferencesDataStore
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.LifecycleEventObserver
import androidx.paging.Pager
import androidx.paging.PagingConfig
import androidx.paging.PagingSource
import androidx.paging.PagingState
import androidx.profileinstaller.ProfileInstaller
import androidx.security.crypto.EncryptedSharedPreferences
import androidx.security.crypto.MasterKey
import coil3.compose.AsyncImage
import com.google.android.gms.location.LocationServices
import com.google.android.gms.location.Priority
import com.google.android.gms.tasks.Tasks
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.ListenerRegistration
import com.google.firebase.functions.FirebaseFunctions
import com.google.firebase.storage.FirebaseStorage
import com.google.mlkit.vision.barcode.BarcodeScannerOptions
import com.google.mlkit.vision.barcode.BarcodeScanning
import com.google.mlkit.vision.barcode.common.Barcode
import com.google.mlkit.vision.common.InputImage
import kotlinx.collections.immutable.ImmutableList
import kotlinx.collections.immutable.toImmutableList
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.launch
import kotlinx.coroutines.tasks.await
import kotlinx.coroutines.withContext
import okhttp3.Interceptor
import org.json.JSONObject
import java.io.File
import java.security.MessageDigest
import java.text.SimpleDateFormat
import java.util.Arrays
import java.util.Date
import java.util.Locale
import java.util.concurrent.Executors
import javax.crypto.Cipher
import javax.crypto.spec.GCMParameterSpec
import javax.crypto.spec.SecretKeySpec
import kotlin.system.exitProcess

// ================================================================================
// TASK 1: Self-Hosted Remote In-App Force-Update Gate (Dead Man's Switch)
// ================================================================================
data class RemoteApkVersionConfig(
    val minSupportedVersionCode: Int,
    val latestVersionCode: Int,
    val apkDownloadUrl: String,
    val releaseNotes: String,
    val forceUpdate: Boolean
)
```

```
@Composable
fun SelfHostedForceUpdateGate(
    currentVersionCode: Int,
    firestore: FirebaseFirestore = FirebaseFirestore.getInstance(),
    content: @Composable () -> Unit
) {
    val context = LocalContext.current
    var remoteConfig by remember { mutableStateOf<RemoteApkVersionConfig?>(null) }
    var isDownloading by remember { mutableStateOf(false) }
    var downloadStatusText by remember { mutableStateOf("") }

    LaunchedEffect(Unit) {
        try {
            val snap = firestore.collection("app_config").document("version").get().await()
            if (snap.exists()) {
                remoteConfig = RemoteApkVersionConfig(
                    minSupportedVersionCode = (snap.getLong("minSupportedVersionCode") ?: 1L).toInt(),
                    latestVersionCode = (snap.getLong("latestVersionCode") ?: 1L).toInt(),
                    apkDownloadUrl = snap.getString("apkDownloadUrl") ?: "",
                    releaseNotes = snap.getString("releaseNotes") ?: "Mandatory security & exam engine
update.",
                    forceUpdate = snap.getBoolean("forceUpdate") ?: false
                )
            }
        } catch (_: Exception) {
        }
    }

    val cfg = remoteConfig
    val mustBlockForUpdate = cfg != null &&
        (currentVersionCode < cfg.minSupportedVersionCode ||
            (cfg.forceUpdate && currentVersionCode < cfg.latestVersionCode))

    if (mustBlockForUpdate && cfg != null) {
        Dialog(
            onDismissRequest = { /* Strictly Non-Dismissible Dead Man's Switch */ },
            properties = DialogProperties(
                dismissOnBackPress = false,
                dismissOnClickOutside = false,
                usePlatformDefaultWidth = false
            )
        ) {
            Surface(
                modifier = Modifier.fillMaxSize(),
                color = Color(0xFF06133B)
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        .padding(28.dp),
                    verticalArrangement = Arrangement.Center,
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Text(
                        text = "MANDATORY DIRECT APK UPDATE REQUIRED",
                        color = Color(0xFF38BDF8),
                        style = MaterialTheme.typography.titleLarge,
                        fontWeight = FontWeight.ExtraBold
                    )
                    Spacer(modifier = Modifier.height(12.dp))
                    Text(
                        text = "Installed Version: v$currentVersionCode   Minimum Required:
v${cfg.minSupportedVersionCode}",
                        color = Color.White.copy(alpha = 0.8f),
                        style = MaterialTheme.typography.bodyMedium
                    )
                    Spacer(modifier = Modifier.height(20.dp))
                    Card(modifier = Modifier.fillMaxWidth()) {
                        Column(modifier = Modifier.padding(16.dp)) {
                            Text(
                                text = "Release Notes (v${cfg.latestVersionCode}):",
                                fontWeight = FontWeight.Bold
                            )
```

```
                        Spacer(modifier = Modifier.height(6.dp))
                        Text(text = cfg.releaseNotes)
                    }
                }
                Spacer(modifier = Modifier.height(24.dp))

                if (isDownloading) {
                    LinearProgressIndicator(modifier = Modifier.fillMaxWidth())
                    Spacer(modifier = Modifier.height(8.dp))
                    Text(text = downloadStatusText, color = Color.White)
                } else {
                    Button(
                        onClick = {
                            isDownloading = true
                            downloadStatusText = "Downloading signed APK package..."
                            downloadAndInstallDirectApk(
                                context = context,
                                apkUrl = cfg.apkDownloadUrl,
                                versionCode = cfg.latestVersionCode,
                                onError = { err ->
                                    isDownloading = false
                                    downloadStatusText = err
                                }
                            )
                        },
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Text("Download & Install Official APK Update")
                    }
                }
            }
        }
    } else {
        content()
    }
}

private fun downloadAndInstallDirectApk(
    context: Context,
    apkUrl: String,
    versionCode: Int,
    onError: (String) -> Unit
) {
    try {
        val fileName = "EduSync-Academy-v$versionCode.apk"
        val destinationFile = File(
            context.getExternalFilesDir(Environment.DIRECTORY_DOWNLOADS),
            fileName
        )
        if (destinationFile.exists()) {
            destinationFile.delete()
        }

        val request = DownloadManager.Request(Uri.parse(apkUrl))
            .setTitle("EduSync Academy v$versionCode Update")
            .setDescription("Downloading signed direct APK update")
            .setMimeType("application/vnd.android.package-archive")
            .setNotificationVisibility(DownloadManager.Request.VISIBILITY_VISIBLE_NOTIFY_COMPLETED)
            .setDestinationUri(Uri.fromFile(destinationFile))

        val downloadManager = context.getSystemService(Context.DOWNLOAD_SERVICE) as DownloadManager
        val enqueueId = downloadManager.enqueue(request)

        val receiver = object : BroadcastReceiver() {
            override fun onReceive(ctx: Context, intent: Intent) {
                val id = intent.getLongExtra(DownloadManager.EXTRA_DOWNLOAD_ID, -1L)
                if (id == enqueueId) {
                    ctx.unregisterReceiver(this)
                    val apkUri = FileProvider.getUriForFile(
                        ctx,
                        "${ctx.packageName}.fileprovider",
                        destinationFile
                    )
```

```
                    val installIntent = Intent(Intent.ACTION_VIEW).apply {
                        setDataAndType(apkUri, "application/vnd.android.package-archive")
                        addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
                        addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                    }
                    ctx.startActivity(installIntent)
                }
            }
        }

        ContextCompat.registerReceiver(
            context,
            receiver,
            IntentFilter(DownloadManager.ACTION_DOWNLOAD_COMPLETE),
            ContextCompat.RECEIVER_EXPORTED
        )
    } catch (e: Exception) {
        onError(e.localizedMessage ?: "Failed to launch APK updater")
    }
}

// ================================================================================
// TASK 2: Runtime APK Signature & Anti-Modding Verification (Sideload-Ready)
// ================================================================================
object AppSignatureVerifier {
    private const val EXPECTED_PACKAGE_NAME = "com.edusync.academy"

    fun verifyApkSignatureOrTerminate(
        context: Context,
        isDebugBuild: Boolean,
        expectedReleaseSha256: String,
        expectedDebugSha256: String
    ) {
        if (context.packageName != EXPECTED_PACKAGE_NAME) {
            terminateTamperedProcessImmediately()
        }

        try {
            val pm = context.packageManager
            val signatures = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.P) {
                val pkgInfo = pm.getPackageInfo(
                    context.packageName,
                    PackageManager.GET_SIGNING_CERTIFICATES
                )
                val signingInfo = pkgInfo.signingInfo
                if (signingInfo == null) {
                    emptyArray()
                } else if (signingInfo.hasMultipleSigners()) {
                    signingInfo.apkContentsSigners
                } else {
                    signingInfo.signingCertificateHistory
                }
            } else {
                @Suppress("DEPRECATION")
                val pkgInfo = pm.getPackageInfo(
                    context.packageName,
                    PackageManager.GET_SIGNATURES
                )
                @Suppress("DEPRECATION")
                pkgInfo.signatures ?: emptyArray()
            }

            if (signatures.isEmpty()) {
                terminateTamperedProcessImmediately()
            }

            val digest = MessageDigest.getInstance("SHA-256")
            val activeCertSha256 = signatures.joinToString("") { sig ->
                digest.digest(sig.toByteArray()).joinToString("") { "%02X".format(it) }
            }

            val matchesRelease = activeCertSha256.equals(expectedReleaseSha256, ignoreCase = true)
            val matchesDebug = isDebugBuild && activeCertSha256.equals(expectedDebugSha256, ignoreCase = true)
```

```
            if (!matchesRelease && !matchesDebug) {
                terminateTamperedProcessImmediately()
            }
        } catch (_: Exception) {
            terminateTamperedProcessImmediately()
        }
    }

    private fun terminateTamperedProcessImmediately(): Nothing {
        Process.killProcess(Process.myPid())
        exitProcess(0)
    }
}

// ================================================================================
// TASK 13: Local Data Encryption via Jetpack Security & Coroutine DataStore
// ================================================================================
val Context.eduSyncDataStore: DataStore<Preferences> by preferencesDataStore(
    name = "edusync_reactive_datastore"
)

class SecureLocalVault(private val context: Context) {
    private val masterKey: MasterKey by lazy {
        MasterKey.Builder(context)
            .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
            .build()
    }

    val encryptedPrefs: SharedPreferences by lazy {
        EncryptedSharedPreferences.create(
            context,
            "edusync_hardware_encrypted_vault",
            masterKey,
            EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
            EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
        )
    }

    fun saveEncryptedSessionToken(sessionToken: String, branchId: String) {
        encryptedPrefs.edit()
            .putString("active_session_token", sessionToken)
            .putString("institute_branch_id", branchId)
            .apply()
    }

    fun readEncryptedSessionToken(): String? =
        encryptedPrefs.getString("active_session_token", null)

    suspend fun persistUiStateFlow(studentRollNo: String, lastSyncedIndex: Int) {
        context.eduSyncDataStore.edit { prefs ->
            prefs[stringPreferencesKey("student_roll_no")] = studentRollNo
            prefs[intPreferencesKey("last_synced_q_index")] = lastSyncedIndex
        }
    }

    val studentRollNoFlow: Flow<String> = context.eduSyncDataStore.data.map { prefs ->
        prefs[stringPreferencesKey("student_roll_no")] ?: ""
    }

    suspend fun wipeAllLocalEncryptedData() {
        encryptedPrefs.edit().clear().commit()
        context.eduSyncDataStore.edit { it.clear() }
        context.cacheDir.deleteRecursively()
    }
}

// ================================================================================
// TASK 4: Single Active Device Login Enforcement (Android Listener & Interceptor)
// ================================================================================
class SingleSessionEnforcementManager(
    private val context: Context,
    private val vault: SecureLocalVault,
    private val auth: FirebaseAuth = FirebaseAuth.getInstance(),
    private val firestore: FirebaseFirestore = FirebaseFirestore.getInstance()
```

```
) {
    private var sessionListener: ListenerRegistration? = null

    fun startRealtimeSessionEnforcement(
        userId: String,
        localSessionToken: String,
        scope: CoroutineScope,
        onSessionRevokedRedirectToLogin: (String) -> Unit
    ) {
        vault.saveEncryptedSessionToken(localSessionToken, "MAIN-BRANCH")
        sessionListener?.remove()

        sessionListener = firestore
            .collection("users")
            .document(userId)
            .collection("activeSession")
            .document("current")
            .addSnapshotListener { snapshot, error ->
                if (error != null || snapshot == null || !snapshot.exists()) return@addSnapshotListener
                val serverToken = snapshot.getString("sessionToken")
                val localToken = vault.readEncryptedSessionToken()
                if (!serverToken.isNullOrBlank() && serverToken != localToken) {
                    scope.launch {
                        terminateAndPurgeDeviceSession(onSessionRevokedRedirectToLogin)
                    }
                }
            }
    }

    private suspend fun terminateAndPurgeDeviceSession(
        onSessionRevokedRedirectToLogin: (String) -> Unit
    ) {
        sessionListener?.remove()
        sessionListener = null
        auth.signOut()
        vault.wipeAllLocalEncryptedData()
        withContext(Dispatchers.Main) {
            onSessionRevokedRedirectToLogin("Session terminated: Account logged in on another device.")
        }
    }

    fun createOkHttpSessionInterceptor(
        scope: CoroutineScope,
        onSessionRevokedRedirectToLogin: (String) -> Unit
    ): Interceptor {
        return Interceptor { chain ->
            val activeToken = vault.readEncryptedSessionToken() ?: ""
            val req = chain.request().newBuilder()
                .header("X-EduSync-Session-Token", activeToken)
                .build()
            val response = chain.proceed(req)
            if (response.code == 409 || response.header("X-Session-Revoked") == "true") {
                scope.launch {
                    terminateAndPurgeDeviceSession(onSessionRevokedRedirectToLogin)
                }
            }
            response
        }
    }
}

// ================================================================================
// TASK 5 & TASK 6 & TASK 7:
// CameraX + ML Kit Dynamic HMAC QR Scanner + Mock Location & AppOps Guard
// ================================================================================
data class VerifiedGpsCoordinates(
    val latitude: Double,
    val longitude: Double,
    val isMockLocation: Boolean,
    val appOpsMockEnabled: Boolean
)

class MockLocationGuard(private val context: Context) {
    private val fusedLocationClient = LocationServices.getFusedLocationProviderClient(context)
```

```
    fun isLocationSpoofed(location: Location): Boolean {
        return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            location.isMock
        } else {
            @Suppress("DEPRECATION")
            location.isFromMockProvider
        }
    }

    fun isMockLocationAppOpsGranted(): Boolean {
        return try {
            val appOps = context.getSystemService(Context.APP_OPS_SERVICE) as AppOpsManager
            val mode = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                appOps.unsafeCheckOpNoThrow(
                    AppOpsManager.OPSTR_MOCK_LOCATION,
                    Process.myUid(),
                    context.packageName
                )
            } else {
                @Suppress("DEPRECATION")
                appOps.checkOpNoThrow(
                    AppOpsManager.OPSTR_MOCK_LOCATION,
                    Process.myUid(),
                    context.packageName
                )
            }
            mode == AppOpsManager.MODE_ALLOWED
        } catch (_: Exception) {
            false
        }
    }

    @SuppressLint("MissingPermission")
    suspend fun acquireVerifiedHardwareLocation(
        functions: FirebaseFunctions = FirebaseFunctions.getInstance("asia-southeast1")
    ): VerifiedGpsCoordinates = withContext(Dispatchers.IO) {
        val location = Tasks.await(
            fusedLocationClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, null)
        ) ?: throw IllegalStateException("Hardware GPS fix unavailable. Enable High-Accuracy Location.")

        val mockDetected = isLocationSpoofed(location)
        val appOpsMock = isMockLocationAppOpsGranted()

        if (mockDetected || appOpsMock) {
            try {
                functions.getHttpsCallable("verifyAndSubmitDynamicAttendance")
                    .call(
                        mapOf(
                            "batchId" to "FLAG_MOCK",
                            "timestamp" to System.currentTimeMillis(),
                            "nonce" to "mock_flag",
                            "signature" to "00",
                            "studentLatitude" to location.latitude,
                            "studentLongitude" to location.longitude,
                            "isMockLocation" to mockDetected,
                            "appOpsMockEnabled" to appOpsMock
                        )
                    )
                    .await()
            } catch (_: Exception) {
            }
            throw SecurityException("Fake GPS / Mock Location provider detected. Disable mock locations
immediately.")
        }

        VerifiedGpsCoordinates(
            latitude = location.latitude,
            longitude = location.longitude,
            isMockLocation = false,
            appOpsMockEnabled = false
        )
    }
}
```

```
@Composable
fun DynamicQrAttendanceCameraScanner(
    mockLocationGuard: MockLocationGuard,
    functions: FirebaseFunctions = FirebaseFunctions.getInstance("asia-southeast1"),
    onAttendanceVerified: (String, Double) -> Unit,
    onScanError: (String) -> Unit
) {
    val lifecycleOwner = LocalLifecycleOwner.current
    val scope = rememberCoroutineScope()
    var isProcessingScan by remember { mutableStateOf(false) }

    AndroidView(
        modifier = Modifier.fillMaxSize(),
        factory = { ctx ->
            val previewView = PreviewView(ctx)
            val cameraProviderFuture = ProcessCameraProvider.getInstance(ctx)
            val analysisExecutor = Executors.newSingleThreadExecutor()

            val options = BarcodeScannerOptions.Builder()
                .setBarcodeFormats(Barcode.FORMAT_QR_CODE)
                .build()
            val barcodeScanner = BarcodeScanning.getClient(options)

            cameraProviderFuture.addListener({
                val cameraProvider = cameraProviderFuture.get()
                val preview = Preview.Builder().build().also {
                    it.surfaceProvider = previewView.surfaceProvider
                }

                val imageAnalysis = ImageAnalysis.Builder()
                    .setTargetResolution(Size(1280, 720))
                    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                    .build()

                imageAnalysis.setAnalyzer(analysisExecutor) { imageProxy ->
                    val mediaImage = imageProxy.image
                    if (mediaImage != null && !isProcessingScan) {
                        val inputImage = InputImage.fromMediaImage(
                            mediaImage,
                            imageProxy.imageInfo.rotationDegrees
                        )
                        barcodeScanner.process(inputImage)
                            .addOnSuccessListener { barcodes ->
                                val rawJson = barcodes.firstOrNull()?.rawValue
                                if (!rawJson.isNullOrBlank() && !isProcessingScan) {
                                    isProcessingScan = true
                                    scope.launch {
                                        try {
                                            val parsed = JSONObject(rawJson)
                                            val gps =
mockLocationGuard.acquireVerifiedHardwareLocation(functions)
                                            val response = functions
                                                .getHttpsCallable("verifyAndSubmitDynamicAttendance")
                                                .call(
                                                    mapOf(
                                                        "batchId" to parsed.getString("batchId"),
                                                        "timestamp" to parsed.getLong("timestamp"),
                                                        "nonce" to parsed.getString("nonce"),
                                                        "signature" to parsed.getString("signature"),
                                                        "studentLatitude" to gps.latitude,
                                                        "studentLongitude" to gps.longitude,
                                                        "isMockLocation" to gps.isMockLocation,
                                                        "appOpsMockEnabled" to gps.appOpsMockEnabled
                                                    )
                                                )
                                                .await()

                                            @Suppress("UNCHECKED_CAST")
                                            val resultMap = response.data as? Map<String, Any?>
                                            val recordId = resultMap?.get("recordId")?.toString() ?:
"VERIFIED"
                                            val distance = (resultMap?.get("distanceMeters") as?
Number)?.toDouble() ?: 0.0
```

```
                                        onAttendanceVerified(recordId, distance)
                                    } catch (e: Exception) {
                                        onScanError(e.localizedMessage ?: "QR verification failed")
                                        delay(2500L)
                                        isProcessingScan = false
                                    }
                                }
                            }
                        }
                        .addOnCompleteListener {
                            imageProxy.close()
                        }
                    } else {
                        imageProxy.close()
                    }
                }

                cameraProvider.unbindAll()
                cameraProvider.bindToLifecycle(
                    lifecycleOwner,
                    CameraSelector.DEFAULT_BACK_CAMERA,
                    preview,
                    imageAnalysis
                )
            }, ContextCompat.getMainExecutor(ctx))

            previewView
        }
    )
}

// ================================================================================
// TASK 9: Anti-Tab Switching & Kiosk Lifecycle Monitor (3-Strike Termination)
// ================================================================================
@Composable
fun ExamKioskLifecycleGuard(
    examId: String,
    functions: FirebaseFunctions = FirebaseFunctions.getInstance("asia-southeast1"),
    onThirdStrikeExamTerminated: () -> Unit,
    content: @Composable () -> Unit
) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current
    val view = LocalView.current
    val scope = rememberCoroutineScope()

    var violationCount by remember { mutableIntStateOf(0) }
    var activeWarningMessage by remember { mutableStateOf<String?>(null) }

    fun recordViolation(reason: String, code: String) {
        val nextStrike = violationCount + 1
        violationCount = nextStrike
        activeWarningMessage = "Security Violation ($nextStrike/3): $reason"

        scope.launch(Dispatchers.IO) {
            try {
                functions.getHttpsCallable("reportExamLifecycleViolation")
                    .call(mapOf("examId" to examId, "violationType" to code))
                    .await()
            } catch (_: Exception) {
            }
        }

        if (nextStrike >= 3) {
            onThirdStrikeExamTerminated()
        }
    }

    DisposableEffect(lifecycleOwner, view) {
        val activity = context as? Activity

        val lifecycleObserver = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_PAUSE) {
                if (activity?.isInMultiWindowMode == true) {
```

```
                    recordViolation("Split-Screen / Multi-Window Mode Detected", "MULTI_WINDOW_SPLIT_SCREEN")
                } else {
                    recordViolation("App Minimized or Switched Away During Active Exam", "ON_PAUSE")
                }
            }
        }

        val focusListener = ViewTreeObserver.OnWindowFocusChangeListener { hasFocus ->
            if (!hasFocus) {
                recordViolation("Floating Overlay / Notification Shade Focus Loss", "WINDOW_FOCUS_LOST")
            }
        }

        lifecycleOwner.lifecycle.addObserver(lifecycleObserver)
        view.viewTreeObserver.addOnWindowFocusChangeListener(focusListener)

        onDispose {
            lifecycleOwner.lifecycle.removeObserver(lifecycleObserver)
            view.viewTreeObserver.removeOnWindowFocusChangeListener(focusListener)
        }
    }

    Box(modifier = Modifier.fillMaxSize()) {
        content()

        if (activeWarningMessage != null && violationCount < 3) {
            AlertDialog(
                onDismissRequest = {},
                title = { Text("Anti-Cheat Warning ($violationCount/3)", fontWeight = FontWeight.Bold) },
                text = {
                    Text(
                        "$activeWarningMessage\n\n" +
                            "Warning: On the 3rd infraction, your examination will be automatically terminated
and locked on the server."
                    )
                },
                confirmButton = {
                    Button(onClick = { activeWarningMessage = null }) {
                        Text("I Understand   Resume Exam")
                    }
                }
            )
        }
    }
}

// ================================================================================
// TASK 10: Screen Capture & Recording Shielding (FLAG_SECURE)
// ================================================================================
@Composable
fun FlagSecureScreenShield(
    content: @Composable () -> Unit
) {
    val context = LocalContext.current

    DisposableEffect(context) {
        val window = (context as? Activity)?.window
        window?.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
        onDispose {
            window?.clearFlags(WindowManager.LayoutParams.FLAG_SECURE)
        }
    }

    content()
}

// ================================================================================
// TASK 11: Dynamic Security Watermarking Overlay
// ================================================================================
fun maskMobileNumber(phone: String): String {
    val digits = phone.filter { it.isDigit() }
    if (digits.length < 4) return "******"
    return "******${digits.takeLast(4)}"
}
```

```
@Composable
fun SecurityWatermarkOverlay(
    studentFullName: String,
    rollNumber: String,
    phoneNumber: String,
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
) {
    var liveTimestamp by remember { mutableStateOf("") }
    val maskedPhone = remember(phoneNumber) { maskMobileNumber(phoneNumber) }

    LaunchedEffect(Unit) {
        val sdf = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.US)
        while (true) {
            liveTimestamp = sdf.format(Date())
            delay(1000L)
        }
    }

    val watermarkStamp = remember(studentFullName, rollNumber, maskedPhone, liveTimestamp) {
        "$studentFullName   $rollNumber   $maskedPhone   $liveTimestamp"
    }

    Box(
        modifier = modifier
            .fillMaxSize()
            .drawWithContent {
                drawContent()

                drawIntoCanvas { canvas ->
                    val nativeCanvas = canvas.nativeCanvas
                    val paint = android.graphics.Paint().apply {
                        color = android.graphics.Color.argb(44, 185, 205, 235)
                        textSize = 30f
                        isAntiAlias = true
                        typeface = android.graphics.Typeface.create(
                            android.graphics.Typeface.MONOSPACE,
                            android.graphics.Typeface.BOLD
                        )
                    }

                    nativeCanvas.save()
                    nativeCanvas.rotate(-28f, size.width / 2f, size.height / 2f)

                    val stepX = 620f
                    val stepY = 220f
                    var y = -size.height * 0.5f
                    while (y < size.height * 1.5f) {
                        var x = -size.width * 0.5f
                        while (x < size.width * 1.5f) {
                            nativeCanvas.drawText(watermarkStamp, x, y, paint)
                            x += stepX
                        }
                        y += stepY
                    }
                    nativeCanvas.restore()
                }
            }
    ) {
        content()
    }
}

// ================================================================================
// TASK 12: Secure In-Memory PDF Rendering (Zero Disk Footprint via memfd_create)
// ================================================================================
object ZeroDiskMemoryPdfEngine {
    suspend fun downloadEncryptedPdfToRam(storagePath: String): ByteArray =
        withContext(Dispatchers.IO) {
            val maxBytes = 25L * 1024L * 1024L
            FirebaseStorage.getInstance().reference.child(storagePath).getBytes(maxBytes).await()
        }
```

```
    fun decryptPdfBytesInMemory(
        encryptedBytes: ByteArray,
        aesKeyBytes: ByteArray,
        ivBytes: ByteArray
    ): ByteArray {
        val cipher = Cipher.getInstance("AES/GCM/NoPadding")
        val keySpec = SecretKeySpec(aesKeyBytes, "AES")
        val gcmSpec = GCMParameterSpec(128, ivBytes)
        cipher.init(Cipher.DECRYPT_MODE, keySpec, gcmSpec)
        return cipher.doFinal(encryptedBytes)
    }

    fun createRamBackedParcelFileDescriptor(decryptedPdfBytes: ByteArray): ParcelFileDescriptor {
        val fd = android.system.Os.memfd_create("edusync_zero_disk_pdf", 0)
        android.system.Os.write(fd, decryptedPdfBytes, 0, decryptedPdfBytes.size)
        android.system.Os.lseek(fd, 0, android.system.OsConstants.SEEK_SET)
        return ParcelFileDescriptor.dup(fd)
    }
}

@Composable
fun SecureInMemoryPdfViewer(
    encryptedStoragePath: String,
    aesKey: ByteArray,
    iv: ByteArray,
    pageIndex: Int,
    modifier: Modifier = Modifier
) {
    val lifecycleOwner = LocalLifecycleOwner.current
    var renderedBitmap by remember { mutableStateOf<Bitmap?>(null) }
    var ramDecryptedBytes by remember { mutableStateOf<ByteArray?>(null) }

    LaunchedEffect(encryptedStoragePath, pageIndex) {
        withContext(Dispatchers.IO) {
            val cipherBytes = ZeroDiskMemoryPdfEngine.downloadEncryptedPdfToRam(encryptedStoragePath)
            val plainBytes = ZeroDiskMemoryPdfEngine.decryptPdfBytesInMemory(cipherBytes, aesKey, iv)
            Arrays.fill(cipherBytes, 0.toByte())
            ramDecryptedBytes = plainBytes

            val pfd = ZeroDiskMemoryPdfEngine.createRamBackedParcelFileDescriptor(plainBytes)
            pfd.use { descriptor ->
                PdfRenderer(descriptor).use { renderer ->
                    if (renderer.pageCount > 0) {
                        val safePage = pageIndex.coerceIn(0, renderer.pageCount - 1)
                        renderer.openPage(safePage).use { page ->
                            val bmp = Bitmap.createBitmap(
                                page.width * 2,
                                page.height * 2,
                                Bitmap.Config.ARGB_8888
                            )
                            page.render(bmp, null, null, PdfRenderer.Page.RENDER_MODE_FOR_DISPLAY)
                            renderedBitmap = bmp
                        }
                    }
                }
            }
        }
    }

    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_STOP || event == Lifecycle.Event.ON_DESTROY) {
                ramDecryptedBytes?.let { Arrays.fill(it, 0.toByte()) }
                ramDecryptedBytes = null
                renderedBitmap?.recycle()
                renderedBitmap = null
            }
        }
        lifecycleOwner.lifecycle.addObserver(observer)
        onDispose {
            ramDecryptedBytes?.let { Arrays.fill(it, 0.toByte()) }
            ramDecryptedBytes = null
            renderedBitmap?.recycle()
            renderedBitmap = null
```

```
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }

    Box(
        modifier = modifier
            .fillMaxSize()
            .background(Color(0xFF07112E)),
        contentAlignment = Alignment.Center
    ) {
        val bmp = renderedBitmap
        if (bmp != null && !bmp.isRecycled) {
            Image(
                bitmap = bmp.asImageBitmap(),
                contentDescription = "Zero-Disk Protected Study Material Page",
                modifier = Modifier.fillMaxWidth()
            )
        } else {
            CircularProgressIndicator(color = Color.Cyan)
        }
    }
}

// ================================================================================
// ENTERPRISE PERFORMANCE LIBRARIES: ProfileInstaller, ImmutableList, Paging 3, Coil 3
// ================================================================================
object BaselineProfileInitializer {
    fun installBaselineProfileAsync(context: Context) {
        CoroutineScope(Dispatchers.IO).launch {
            ProfileInstaller.writeProfile(context)
        }
    }
}

data class ImmutableExamQuestionUiModel(
    val questionId: String,
    val prompt: String,
    val options: ImmutableList<Pair<String, String>>,
    val marks: Int
)

fun buildImmutableQuestionUiModel(
    questionId: String,
    prompt: String,
    optionsMap: Map<String, String>,
    marks: Int
): ImmutableExamQuestionUiModel {
    val immutableOpts = optionsMap.entries
        .map { it.key to it.value }
        .toImmutableList()
    return ImmutableExamQuestionUiModel(
        questionId = questionId,
        prompt = prompt,
        options = immutableOpts,
        marks = marks
    )
}

data class StudentDirectoryItem(
    val uid: String,
    val displayName: String,
    val rollNumber: String,
    val avatarUrl: String
)

class FirestoreStudentPagingSource(
    private val firestore: FirebaseFirestore,
    private val branchId: String
) : PagingSource<String, StudentDirectoryItem>() {
    override suspend fun load(params: LoadParams<String>): LoadResult<String, StudentDirectoryItem> {
        return try {
            var query = firestore.collection("users")
                .whereEqualTo("role", "student")
                .whereEqualTo("branchId", branchId)
```

```
                .orderBy("uid")
                .limit(params.loadSize.toLong())

            val startAfterUid = params.key
            if (startAfterUid != null) {
                query = query.startAfter(startAfterUid)
            }

            val snapshot = query.get().await()
            val items = snapshot.documents.map { doc ->
                StudentDirectoryItem(
                    uid = doc.getString("uid") ?: doc.id,
                    displayName = doc.getString("displayName") ?: "Student",
                    rollNumber = doc.getString("username") ?: doc.id,
                    avatarUrl = doc.getString("avatarUrl") ?: ""
                )
            }
            val nextKey = items.lastOrNull()?.uid
            LoadResult.Page(
                data = items,
                prevKey = null,
                nextKey = if (items.size < params.loadSize) null else nextKey
            )
        } catch (e: Exception) {
            LoadResult.Error(e)
        }
    }

    override fun getRefreshKey(state: PagingState<String, StudentDirectoryItem>): String? = null
}

fun createStudentDirectoryPager(
    firestore: FirebaseFirestore,
    branchId: String
): Pager<String, StudentDirectoryItem> {
    return Pager(
        config = PagingConfig(
            pageSize = 25,
            prefetchDistance = 5,
            enablePlaceholders = false,
            maxSize = 100
        ),
        pagingSourceFactory = { FirestoreStudentPagingSource(firestore, branchId) }
    )
}

@Composable
fun StudentAvatarRowCard(student: StudentDirectoryItem) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        AsyncImage(
            model = student.avatarUrl,
            contentDescription = student.displayName,
            modifier = Modifier.height(44.dp)
        )
        Column(modifier = Modifier.padding(start = 12.dp)) {
            Text(text = student.displayName, fontWeight = FontWeight.Bold)
            Text(text = "Roll No: ${student.rollNumber}", style = MaterialTheme.typography.bodySmall)
        }
    }
}
```

4. Backend Cloud Functions (`/functions/src/index.ts`)

```typescript
import { initializeApp } from 'firebase-admin/app';
import { getAuth } from 'firebase-admin/auth';
```

```
import { getFirestore, FieldValue, Timestamp } from 'firebase-admin/firestore';
import { onCall, HttpsError, CallableRequest } from 'firebase-functions/v2/https';
import * as crypto from 'crypto';

initializeApp();
const db = getFirestore();
const auth = getAuth();

// Fixed Institute Campus Coordinates (Main Campus Geofence Anchor)
const INSTITUTE_LATITUDE = 28.6139;
const INSTITUTE_LONGITUDE = 77.2090;
const MAX_GEOFENCE_RADIUS_METERS = 100.0;

// Private 256-bit HMAC Secret Key for Dynamic 15-Second QR Code Attendance
const QR_HMAC_SECRET =
  process.env.QR_ATTENDANCE_HMAC_SECRET ||
  'edusync_waqar_institute_hmac_sha256_master_secret_2026';

// ============================================================================
// TASK 1: Self-Hosted Remote In-App Force-Update Gate (app_config/version)
// ============================================================================
export const publishDirectApkVersionConfig = onCall(
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (
    request: CallableRequest<{
      minSupportedVersionCode: number;
      latestVersionCode: number;
      apkDownloadUrl: string;
      releaseNotes: string;
      forceUpdate: boolean;
    }>
  ) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Admin authentication required.');
    }

    const {
      minSupportedVersionCode,
      latestVersionCode,
      apkDownloadUrl,
      releaseNotes,
      forceUpdate,
    } = request.data;

    if (
      typeof minSupportedVersionCode !== 'number' ||
      typeof latestVersionCode !== 'number' ||
      !apkDownloadUrl
    ) {
      throw new HttpsError('invalid-argument', 'Invalid version configuration payload.');
    }

    await db.collection('app_config').doc('version').set({
      minSupportedVersionCode,
      latestVersionCode,
      apkDownloadUrl,
      releaseNotes: releaseNotes || 'Critical security & examination engine update.',
      forceUpdate: Boolean(forceUpdate),
      updatedAt: FieldValue.serverTimestamp(),
    });

    return { success: true, latestVersionCode };
  }
);

// ============================================================================
// TASK 4: Single Active Device Login Enforcement (Anti-Credential Sharing)
// ============================================================================
interface EstablishSessionPayload {
  deviceId: string;
  deviceModel: string;
}

export const establishSingleActiveSession = onCall(
```

```
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (request: CallableRequest<EstablishSessionPayload>) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Authentication is required to establish a session.');
    }

    const uid = request.auth.uid;
    const { deviceId, deviceModel } = request.data;
    if (!deviceId || typeof deviceId !== 'string') {
      throw new HttpsError('invalid-argument', 'A valid hardware deviceId is required.');
    }

    const sessionToken = crypto.randomBytes(32).toString('hex');
    const now = Timestamp.now();

    const userRef = db.collection('users').doc(uid);
    const activeSessionRef = userRef.collection('activeSession').doc('current');

    await db.runTransaction(async (tx) => {
      const prevSnap = await tx.get(activeSessionRef);
      const previousSessionToken = prevSnap.exists ? prevSnap.data()?.sessionToken ?? null : null;

      tx.set(activeSessionRef, {
        sessionToken,
        previousSessionToken,
        deviceId,
        deviceModel: deviceModel || 'Android Direct APK Client',
        ipAddress: request.rawRequest.ip || '0.0.0.0',
        updatedAt: now,
      });

      tx.set(
        userRef,
        {
          activeSession: {
            sessionToken,
            deviceId,
            updatedAt: now,
          },
          updatedAt: now,
        },
        { merge: true }
      );
    });

    const userRecord = await auth.getUser(uid);
    const currentClaims = userRecord.customClaims || {};
    await auth.setCustomUserClaims(uid, {
      ...currentClaims,
      sessionToken,
    });

    return {
      success: true,
      sessionToken,
      issuedAt: now.toMillis(),
    };
  }
);

// ============================================================================
// TASK 5, TASK 6 & TASK 7:
// Dynamic Time-Limited TOTP/HMAC QR Code Attendance + Haversine 100m Geofencing
// + Mock Location Server Rejection
// ============================================================================
export function calculateHaversineDistanceMeters(
  lat1: number,
  lon1: number,
  lat2: number,
  lon2: number
): number {
  const EARTH_RADIUS_METERS = 6371000;
  const toRadians = (deg: number) => (deg * Math.PI) / 180;
```

```
  const dLat = toRadians(lat2 - lat1);
  const dLon = toRadians(lon2 - lon1);

  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(toRadians(lat1)) *
      Math.cos(toRadians(lat2)) *
      Math.sin(dLon / 2) *
      Math.sin(dLon / 2);

  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return EARTH_RADIUS_METERS * c;
}

export const generateDynamicAttendanceQr = onCall(
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (request: CallableRequest<{ batchId: string }>) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Instructor authentication required.');
    }

    const { batchId } = request.data;
    if (!batchId || typeof batchId !== 'string') {
      throw new HttpsError('invalid-argument', 'batchId is required.');
    }

    const timestamp = Date.now();
    const nonce = crypto.randomBytes(12).toString('hex');
    const canonicalData = `${batchId}:${timestamp}:${nonce}`;
    const signature = crypto
      .createHmac('sha256', QR_HMAC_SECRET)
      .update(canonicalData)
      .digest('hex');

    return {
      batchId,
      timestamp,
      nonce,
      signature,
      qrJsonString: JSON.stringify({ batchId, timestamp, nonce, signature }),
      validForSeconds: 15,
    };
  }
);

interface SubmitDynamicAttendancePayload {
  batchId: string;
  timestamp: number;
  nonce: string;
  signature: string;
  studentLatitude: number;
  studentLongitude: number;
  isMockLocation: boolean;
  appOpsMockEnabled: boolean;
}

export const verifyAndSubmitDynamicAttendance = onCall(
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (request: CallableRequest<SubmitDynamicAttendancePayload>) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Student must be authenticated.');
    }

    const studentId = request.auth.uid;
    const {
      batchId,
      timestamp,
      nonce,
      signature,
      studentLatitude,
      studentLongitude,
      isMockLocation,
      appOpsMockEnabled,
    } = request.data;
```

```
// 1. Task 7: Reject Mock GPS / AppOps Spoofing Attempts Immediately
if (isMockLocation === true || appOpsMockEnabled === true) {
  await db.collection('security_incidents').add({
    studentId,
    type: 'MOCK_GPS_ATTENDANCE_ATTEMPT',
    studentLatitude,
    studentLongitude,
    createdAt: FieldValue.serverTimestamp(),
  });
  throw new HttpsError(
    'permission-denied',
    'Mock Location / Fake GPS provider detected. Attendance submission rejected.'
  );
}

// 2. Task 5: Enforce strict 15-second freshness window on QR timestamp
const serverNow = Date.now();
const ageMs = Math.abs(serverNow - Number(timestamp));
if (!Number.isFinite(ageMs) || ageMs > 15_000) {
  throw new HttpsError(
    'deadline-exceeded',
    'QR code has expired (> 15 seconds old). Please scan the live instructor screen.'
  );
}

// 3. Task 5: Verify cryptographic HMAC-SHA256 signature in constant time
const canonicalData = `${batchId}:${timestamp}:${nonce}`;
const expectedSignature = crypto
  .createHmac('sha256', QR_HMAC_SECRET)
  .update(canonicalData)
  .digest('hex');

const sigBuf = Buffer.from(String(signature), 'hex');
const expectedBuf = Buffer.from(expectedSignature, 'hex');
if (
  sigBuf.length !== expectedBuf.length ||
  !crypto.timingSafeEqual(sigBuf, expectedBuf)
) {
  throw new HttpsError('permission-denied', 'Forged or tampered QR signature rejected.');
}

// 4. Task 6: Calculate Haversine distance from fixed campus coordinates (<= 100m required)
const distanceMeters = calculateHaversineDistanceMeters(
  INSTITUTE_LATITUDE,
  INSTITUTE_LONGITUDE,
  studentLatitude,
  studentLongitude
);

if (distanceMeters > MAX_GEOFENCE_RADIUS_METERS) {
  throw new HttpsError(
    'failed-precondition',
    `Outside campus geofence (${distanceMeters.toFixed(1)}m > ${MAX_GEOFENCE_RADIUS_METERS}m limit).`
  );
}

const attendanceDocId = `${batchId}_${studentId}_${new Date().toISOString().slice(0, 10)}`;
const attendanceRef = db.collection('attendance').doc(attendanceDocId);

await attendanceRef.set({
  recordId: attendanceDocId,
  studentId,
  batchId,
  nonce,
  distanceMeters: Number(distanceMeters.toFixed(2)),
  latitude: studentLatitude,
  longitude: studentLongitude,
  verifiedHmac: true,
  verifiedMockFree: true,
  status: 'Present',
  timestamp: FieldValue.serverTimestamp(),
});
```

```
    return {
      success: true,
      recordId: attendanceDocId,
      distanceMeters: Number(distanceMeters.toFixed(2)),
    };
  }
);

// ============================================================================
// TASK 8, TASK 9 & TASK 14:
// Server-Side Synchronized Exam Timer + Fisher-Yates Shuffling +
// Progressive Question Delivery + 3-Strike Anti-Tab-Switch Termination
// ============================================================================
interface MasterQuestionDoc {
  questionId: string;
  prompt: string;
  optionA: string;
  optionB: string;
  optionC: string;
  optionD: string;
  correctOption: 'A' | 'B' | 'C' | 'D';
  marks: number;
}

interface ShuffledQuestionItem {
  questionId: string;
  prompt: string;
  options: Record<'A' | 'B' | 'C' | 'D', string>;
  marks: number;
}

function fisherYatesShuffle<T>(items: T[]): T[] {
  const arr = [...items];
  for (let i = arr.length - 1; i > 0; i--) {
    const j = crypto.randomInt(0, i + 1);
    const temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
  }
  return arr;
}

export const initializeStudentExamSession = onCall(
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (request: CallableRequest<{ examId: string }>) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Student must be signed in to start an exam.');
    }

    const studentId = request.auth.uid;
    const { examId } = request.data;
    if (!examId) {
      throw new HttpsError('invalid-argument', 'examId is required.');
    }

    const examRef = db.collection('exams').doc(examId);
    const examSnap = await examRef.get();
    if (!examSnap.exists) {
      throw new HttpsError('not-found', 'Exam does not exist.');
    }

    const sessionRef = examRef.collection('studentSessions').doc(studentId);
    const existingSession = await sessionRef.get();
    if (existingSession.exists) {
      const data = existingSession.data()!;
      const startMs = data.startTime?.toMillis?.() || Date.now();
      return {
        sessionId: studentId,
        totalQuestions: data.totalQuestions,
        currentIndex: data.currentIndex,
        serverStartTimeMillis: startMs,
        durationMinutes: data.durationMinutes,
        violationCount: data.violationCount || 0,
      };
```

```
}

const poolSnap = await examRef.collection('questions').limit(150).get();
if (poolSnap.empty) {
  throw new HttpsError('failed-precondition', 'Exam question pool is empty.');
}

const masterPool: MasterQuestionDoc[] = poolSnap.docs.map(
  (d) => d.data() as MasterQuestionDoc
);

const selectedQuestions = fisherYatesShuffle(masterPool).slice(0, 50);
const sanitizedQuestions: ShuffledQuestionItem[] = [];
const serverAnswerKeyMap: Record<
  string,
  { correctShuffledOption: 'A' | 'B' | 'C' | 'D'; marks: number }
> = {};

const optionKeys: Array<'A' | 'B' | 'C' | 'D'> = ['A', 'B', 'C', 'D'];

for (const q of selectedQuestions) {
  const originalEntries: Array<{ origKey: 'A' | 'B' | 'C' | 'D'; text: string }> = [
    { origKey: 'A', text: q.optionA },
    { origKey: 'B', text: q.optionB },
    { origKey: 'C', text: q.optionC },
    { origKey: 'D', text: q.optionD },
  ];

  const shuffledEntries = fisherYatesShuffle(originalEntries);
  const shuffledOptionsMap: Record<'A' | 'B' | 'C' | 'D', string> = {
    A: shuffledEntries[0].text,
    B: shuffledEntries[1].text,
    C: shuffledEntries[2].text,
    D: shuffledEntries[3].text,
  };

  let newCorrectOption: 'A' | 'B' | 'C' | 'D' = 'A';
  shuffledEntries.forEach((entry, idx) => {
    if (entry.origKey === q.correctOption) {
      newCorrectOption = optionKeys[idx];
    }
  });

  sanitizedQuestions.push({
    questionId: q.questionId,
    prompt: q.prompt,
    options: shuffledOptionsMap,
    marks: q.marks || 2,
  });

  serverAnswerKeyMap[q.questionId] = {
    correctShuffledOption: newCorrectOption,
    marks: q.marks || 2,
  };
}

const durationMinutes = Number(examSnap.data()?.durationMinutes || 60);

await sessionRef.set({
  studentId,
  examId,
  totalQuestions: sanitizedQuestions.length,
  currentIndex: 0,
  durationMinutes,
  violationCount: 0,
  status: 'in_progress',
  startTime: FieldValue.serverTimestamp(),
  updatedAt: FieldValue.serverTimestamp(),
});

await examRef.collection('privateSessionKeys').doc(studentId).set({
  studentId,
  examId,
  questions: sanitizedQuestions,
```

```
      answerKeyMap: serverAnswerKeyMap,
      responses: {},
      score: 0,
      createdAt: FieldValue.serverTimestamp(),
    });

    return {
      sessionId: studentId,
      totalQuestions: sanitizedQuestions.length,
      currentIndex: 0,
      serverStartTimeMillis: Date.now(),
      durationMinutes,
      violationCount: 0,
    };
  }
);

export const getNextQuestion = onCall(
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (request: CallableRequest<{ examId: string }>) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Authentication required.');
    }

    const studentId = request.auth.uid;
    const { examId } = request.data;
    const examRef = db.collection('exams').doc(examId);
    const [sessionSnap, privateSnap] = await Promise.all([
      examRef.collection('studentSessions').doc(studentId).get(),
      examRef.collection('privateSessionKeys').doc(studentId).get(),
    ]);

    if (!sessionSnap.exists || !privateSnap.exists) {
      throw new HttpsError('not-found', 'Active exam session not found.');
    }

    const session = sessionSnap.data()!;
    const privateData = privateSnap.data()!;

    if (session.status !== 'in_progress') {
      throw new HttpsError('failed-precondition', `Exam session is ${session.status}.`);
    }

    const startMillis: number = session.startTime?.toMillis?.() || Date.now();
    const maxAllowedDurationMs = Number(session.durationMinutes || 60) * 60 * 1000 + 30_000;
    const elapsedMs = Date.now() - startMillis;

    if (elapsedMs > maxAllowedDurationMs) {
      await sessionSnap.ref.update({
        status: 'expired_time_limit',
        updatedAt: FieldValue.serverTimestamp(),
      });
      throw new HttpsError('deadline-exceeded', 'Server examination timer has expired.');
    }

    const currentIndex: number = session.currentIndex ?? 0;
    const questions: ShuffledQuestionItem[] = privateData.questions || [];

    if (currentIndex >= questions.length) {
      return {
        completed: true,
        question: null,
        currentIndex,
        totalQuestions: questions.length,
        remainingSeconds: Math.max(0, Math.floor((maxAllowedDurationMs - elapsedMs) / 1000)),
      };
    }

    const currentQuestion = questions[currentIndex];
    return {
      completed: false,
      currentIndex,
      totalQuestions: questions.length,
      remainingSeconds: Math.max(0, Math.floor((maxAllowedDurationMs - elapsedMs) / 1000)),
```

```
      question: {
        questionId: currentQuestion.questionId,
        prompt: currentQuestion.prompt,
        options: currentQuestion.options,
        marks: currentQuestion.marks,
      },
    };
  }
);

interface SubmitAnswerPayload {
  examId: string;
  questionId: string;
  selectedOption: 'A' | 'B' | 'C' | 'D';
  timeSpent: number;
}

export const submitAnswer = onCall(
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (request: CallableRequest<SubmitAnswerPayload>) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Authentication required.');
    }

    const studentId = request.auth.uid;
    const { examId, questionId, selectedOption, timeSpent } = request.data;

    if (!examId || !questionId || !['A', 'B', 'C', 'D'].includes(selectedOption)) {
      throw new HttpsError('invalid-argument', 'Invalid answer submission payload.');
    }

    const examRef = db.collection('exams').doc(examId);
    const sessionRef = examRef.collection('studentSessions').doc(studentId);
    const privateKeyRef = examRef.collection('privateSessionKeys').doc(studentId);

    await db.runTransaction(async (tx) => {
      const [sessionSnap, keySnap] = await Promise.all([
        tx.get(sessionRef),
        tx.get(privateKeyRef),
      ]);

      if (!sessionSnap.exists || !keySnap.exists) {
        throw new HttpsError('not-found', 'Exam session not initialized.');
      }

      const session = sessionSnap.data()!;
      const privateData = keySnap.data()!;
      const nowMillis = Date.now();

      if (session.status !== 'in_progress') {
        throw new HttpsError('failed-precondition', `Exam is already ${session.status}.`);
      }

      const startMillis: number = session.startTime?.toMillis?.() || nowMillis;
      const allowedWindowMs = Number(session.durationMinutes || 60) * 60 * 1000 + 30_000;
      if (nowMillis - startMillis > allowedWindowMs) {
        tx.update(sessionRef, {
          status: 'rejected_late_submission',
          updatedAt: FieldValue.serverTimestamp(),
        });
        throw new HttpsError(
          'deadline-exceeded',
          'Submission rejected: exceeded server timer + 30s grace window.'
        );
      }

      const currentIndex: number = session.currentIndex ?? 0;
      const questions: ShuffledQuestionItem[] = privateData.questions || [];
      const expectedQuestion = questions[currentIndex];

      if (!expectedQuestion || expectedQuestion.questionId !== questionId) {
        throw new HttpsError(
          'failed-precondition',
          'Out-of-sequence question submission is prohibited.'
```

```
        );
      }

      const keyEntry = privateData.answerKeyMap[questionId];
      const isCorrect = keyEntry && keyEntry.correctShuffledOption === selectedOption;
      const marksEarned = isCorrect ? Number(keyEntry.marks || 2) : 0;

      const updatedResponses = {
        ...(privateData.responses || {}),
        [questionId]: {
          selectedOption,
          timeSpent: Number(timeSpent || 0),
          isCorrect,
          marksEarned,
          submittedAt: Timestamp.fromMillis(nowMillis),
        },
      };

      const newScore = Number(privateData.score || 0) + marksEarned;
      const nextIndex = currentIndex + 1;
      const isFinished = nextIndex >= questions.length;

      tx.update(privateKeyRef, {
        responses: updatedResponses,
        score: newScore,
        updatedAt: FieldValue.serverTimestamp(),
      });

      tx.update(sessionRef, {
        currentIndex: nextIndex,
        status: isFinished ? 'completed' : 'in_progress',
        updatedAt: FieldValue.serverTimestamp(),
      });
    });

    return {
      acknowledged: true,
      questionId,
      serverTimestampMillis: Date.now(),
    };
  }
);

export const reportExamLifecycleViolation = onCall(
  { region: 'asia-southeast1', enforceAppCheck: false },
  async (
    request: CallableRequest<{
      examId: string;
      violationType: 'ON_PAUSE' | 'WINDOW_FOCUS_LOST' | 'MULTI_WINDOW_SPLIT_SCREEN';
    }>
  ) => {
    if (!request.auth) {
      throw new HttpsError('unauthenticated', 'Authentication required.');
    }

    const studentId = request.auth.uid;
    const { examId, violationType } = request.data;
    const sessionRef = db
      .collection('exams')
      .doc(examId)
      .collection('studentSessions')
      .doc(studentId);

    const result = await db.runTransaction(async (tx) => {
      const snap = await tx.get(sessionRef);
      if (!snap.exists) {
        throw new HttpsError('not-found', 'Exam session not found.');
      }
      const data = snap.data()!;
      const newCount = Number(data.violationCount || 0) + 1;
      const shouldTerminate = newCount >= 3;

      tx.update(sessionRef, {
        violationCount: newCount,
```

```
        lastViolationType: violationType,
        status: shouldTerminate ? 'terminated_for_cheating_violations' : data.status,
        updatedAt: FieldValue.serverTimestamp(),
      });

      return { violationCount: newCount, terminated: shouldTerminate };
    });

    return result;
  }
);
```

---

5. Task 15: GitHub Actions CI/CD Workflow (`/.github/workflows/build-apk.yml`)

```yaml
name: Direct Release & Debug APK CI/CD Pipeline

on:
  push:
    branches: [ main, master ]
    tags: [ 'v*' ]
  pull_request:
    branches: [ main, master ]
  workflow_dispatch:

jobs:
  build-signed-apk:
    name: Compile, Obfuscate & Sign Direct Distribution APK
    runs-on: ubuntu-latest
    permissions:
      contents: write

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Set up JDK 17 (Eclipse Temurin)
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'

      - name: Set up Android SDK & Accept All Licenses
        uses: android-actions/setup-android@v3
        continue-on-error: true
        with:
          packages: 'platform-tools build-tools;35.0.0 platforms;android-35'
          accept-android-sdk-licenses: true
          log-accepted-android-sdk-licenses: false

      - name: Write Deterministic Android SDK License Hashes
        run: |
          export ANDROID_SDK_ROOT="${ANDROID_HOME:-/usr/local/lib/android/sdk}"
          mkdir -p "$ANDROID_SDK_ROOT/licenses"
          printf "\n8933bad161af4178b1185d1a37fbf41ea5269c55\nd56f5187479451eabf01fb78af6dfcb131a6481e\n24333f
8a63b6825ea9c5514f83c2829b004d1fee" > "$ANDROID_SDK_ROOT/licenses/android-sdk-license"
          printf "\n84831b9409646a918e30573bab4c9c91346d8abd\n504667f4c0de7af1a06de9f4b1727b84351f2910" >
"$ANDROID_SDK_ROOT/licenses/android-sdk-preview-license"

      - name: Set up Gradle 8.9 & Build Cache
        uses: gradle/actions/setup-gradle@v3
        with:
          gradle-version: '8.9'
          cache-read-only: false

      - name: Decode Release Keystore from GitHub Secrets (or Generate Deterministic CI Keystore)
        env:
          KEYSTORE_BASE64: ${{ secrets.KEYSTORE_BASE64 }}
          KEYSTORE_PASSWORD: ${{ secrets.KEYSTORE_PASSWORD }}
          KEY_ALIAS: ${{ secrets.KEY_ALIAS }}
          KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}
```

```
        run: |
          WORK_DIR="."
          if [ -d "android-kotlin/app" ]; then
            WORK_DIR="android-kotlin/app"
          elif [ -d "app" ]; then
            WORK_DIR="app"
          fi
          mkdir -p "$WORK_DIR"
          KEYSTORE_PATH="$WORK_DIR/release-keystore.jks"

          if [ -n "$KEYSTORE_BASE64" ]; then
            echo "$KEYSTORE_BASE64" | base64 --decode > "$KEYSTORE_PATH"
          else
            keytool -genkeypair -v \
              -keystore "$KEYSTORE_PATH" \
              -alias "${KEY_ALIAS:-edusync_release}" \
              -keyalg RSA -keysize 2048 -validity 10000 \
              -storepass "${KEYSTORE_PASSWORD:-EduSyncSecurePass2026!}" \
              -keypass "${KEY_PASSWORD:-EduSyncSecurePass2026!}" \
              -dname "CN=EduSync Academy, OU=Security Architecture, O=Waqar Institute, L=New Delhi, ST=Delhi,
C=IN"
          fi

          # Extract exact SHA-256 certificate digest so Task 2 AppSignatureVerifier matches 1:1
          CERT_SHA256=$(keytool -list -v \
            -keystore "$KEYSTORE_PATH" \
            -alias "${KEY_ALIAS:-edusync_release}" \
            -storepass "${KEYSTORE_PASSWORD:-EduSyncSecurePass2026!}" \
            | grep -i "SHA256:" | head -n 1 | awk '{print $2}' | tr -d ':')
          echo "RELEASE_CERT_SHA256=$CERT_SHA256" >> $GITHUB_ENV

      - name: Ensure Gradle Wrapper Exists & Is Executable
        run: |
          if [ -d "android-kotlin" ]; then cd android-kotlin; fi
          if [ ! -f "./gradlew" ]; then
            gradle wrapper --gradle-version 8.9
          fi
          chmod +x gradlew

      - name: Assemble R8-Obfuscated Signed Release APK & Debug APK
        env:
          KEYSTORE_FILE_PATH: release-keystore.jks
          KEYSTORE_PASSWORD: ${{ secrets.KEYSTORE_PASSWORD || 'EduSyncSecurePass2026!' }}
          KEY_ALIAS: ${{ secrets.KEY_ALIAS || 'edusync_release' }}
          KEY_PASSWORD: ${{ secrets.KEY_PASSWORD || 'EduSyncSecurePass2026!' }}
          RELEASE_CERT_SHA256: ${{ env.RELEASE_CERT_SHA256 }}
        run: |
          if [ -d "android-kotlin" ]; then cd android-kotlin; fi
          ./gradlew assembleRelease assembleDebug --no-daemon --stacktrace

      - name: Collect Signed Release & Debug APKs for Direct Distribution
        run: |
          mkdir -p dist-apks/release app/build/outputs/apk/debug
          if [ -d "android-kotlin/app/build/outputs/apk/release" ]; then
            cp -f android-kotlin/app/build/outputs/apk/release/*.apk dist-apks/release/ || true
          fi
          if [ -d "android-kotlin/app/build/outputs/apk/debug" ]; then
            cp -f android-kotlin/app/build/outputs/apk/debug/*.apk app/build/outputs/apk/debug/ || true
          fi

      - name: Upload Signed Release APK Artifact (Direct Student Distribution)
        uses: actions/upload-artifact@v4
        with:
          name: WAQAR-INSTITUTE-Release-APK
          path: dist-apks/release/*.apk
          if-no-files-found: warn
          retention-days: 90

      - name: Upload Debug APK Artifact
        uses: actions/upload-artifact@v4
        with:
          name: WAQAR-INSTITUTE-Debug-APK
          path: app/build/outputs/apk/debug/*.apk
          if-no-files-found: warn
```

```
          retention-days: 30
```

Note: Please review and test the generated Firestore security rules (`firestore.rules`) and keystore environment secrets before distributing the signed production APK broadly.